

Automated Credit Card Approval Prediction System

Project Description:

The **Automated Credit Card Approval Prediction System** is a Machine Learning-based web application that helps banks and financial institutions predict whether a customer's credit card application should be **Approved** or **Rejected** based on personal and financial information.

The system analyzes applicant details such as **Age, Annual Income, Employment Status, Work Experience, Education Level, Marital Status, Number of Dependents, Existing Debt, Credit Score, and Loan Amount** using a trained Machine Learning model.

The application is developed using **Python, Flask, Scikit-learn, Pandas, and NumPy**. A trained **Random Forest Classifier** (or your best-performing model) processes the applicant's information and provides instant approval predictions. The user-friendly web application consists of **Home, About, Prediction, and Result** pages that allow users to submit their details and receive immediate results.

The application can be deployed on **Render**, making it accessible through any web browser without additional software installation.

- **Live URL:** <https://creditcardapproval.onrender.com>
- **Local URL:** <http://localhost:5000>

The project assists banks in reducing manual verification time, improving decision accuracy, and providing a faster credit card approval process.

Scenarios

Scenario 1: Credit Card Approval Prediction

An applicant enters personal and financial information through the prediction form. The Machine Learning model analyzes the provided details and predicts whether the credit card application is likely to be Approved or Rejected.

Scenario 2: First-Time User

A new user accesses the deployed Flask application through a web browser. After entering the required applicant information on the Prediction page, the application instantly displays the approval prediction without requiring registration or login.

Scenario 3: Decision Support for Better Farming

A bank employee evaluates multiple customer applications using the system. The application predicts approval status for each applicant, helping the bank make faster and more consistent credit approval decisions.

Technical Architecture :

The Credit Card Approval Prediction System is developed using **Flask** as the web application framework and **Scikit-learn** for Machine Learning. Users submit applicant information through the Flask interface. The application preprocesses the input data using the same **StandardScaler** and encoding techniques applied during model training. The processed data is then passed to the trained **Random Forest** model (`credit_card_model.pkl`) to predict whether the application should be Approved or Rejected. The prediction result is displayed instantly on the Result page

Architecture Diagram:

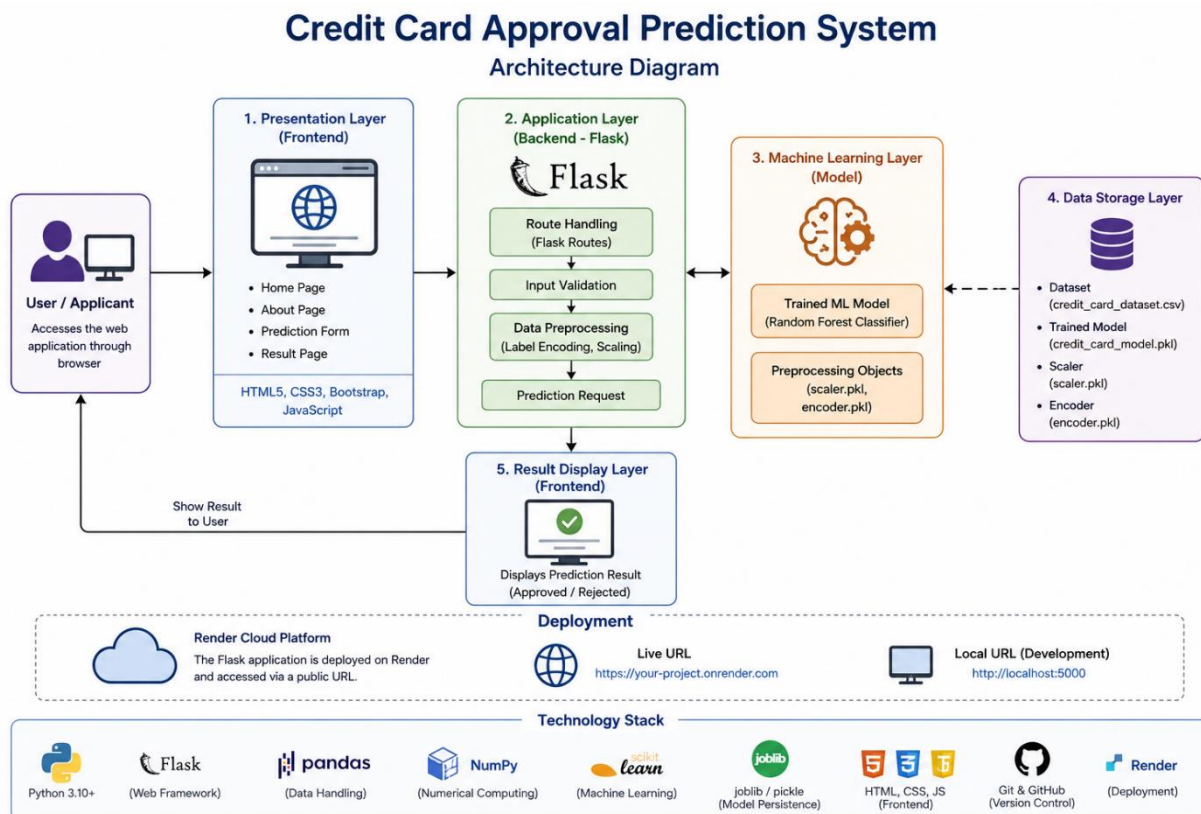


Figure 1.1: Data flow from browser input to creditcard recommendation.

Prerequisites & Technology Stack

- Python 3.10 or above
- Flask — web application framework
- Pandas, NumPy — data handling
- Matplotlib, Seaborn — visualization
- Scikit-learn — machine learning (KMeans, LogisticRegression, StandardScaler)
- Pickle — model persistence
- Git & GitHub — version control
- Render — cloud deployment
- Internet connection (for accessing the deployed application)

Dataset Collection & Environment Setup

Dataset Source

- Credit Card Approval Dataset (Kaggle / UCI Repository)

Dataset Features

- Age
- Annual Income
- Employment Status
- Work Experience
- Education Level
- Marital Status
- Number of Dependents
- Existing Debt
- Credit Score
- Loan Amount

Target

- Credit Card Status (Approved / Rejected)

Environment Setup

- Install Python

- Create Virtual Environment
- Install Dependencies using

`pip install -r requirements.txt`

- Load dataset using Pandas
- Verify missing values
- Train Machine Learning model

Exploratory Data Analysis:

Univariate Analysis

Each applicant feature was analyzed individually to understand its distribution, value ranges, and data quality before training the Machine Learning model.

Distribution of Applicant Features – Credit Card Approval Dataset

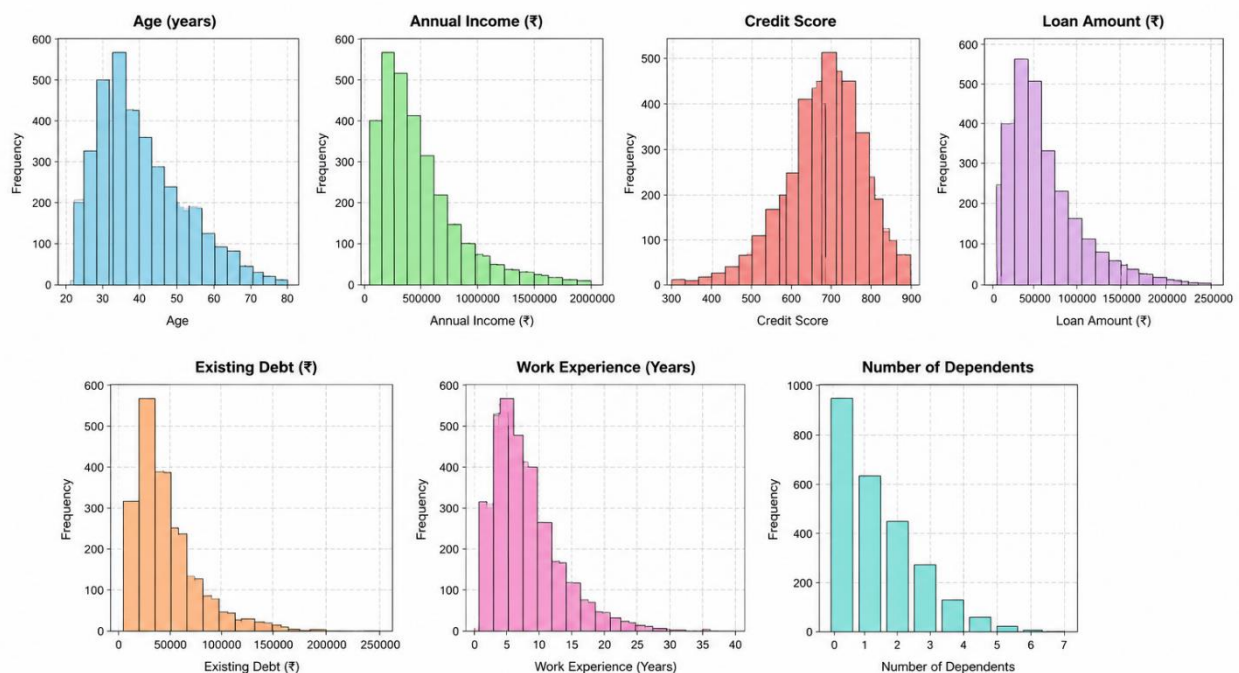


Figure 2.1: Distribution of all 7 applicant features.

Bivariate Analysis

Relationships between applicant income, credit score, debt, employment status, and approval status were analyzed to identify important predictive features.

Multivariate Analysis

Correlation analysis and feature relationships were examined to understand how multiple applicant attributes influence credit card approval decisions.

Figure 2.2: Credit Score vs Approval Status – Class-wise Distribution

The plot below shows the distribution of Credit Score for customers who were Approved and Rejected. Approved applicants generally have higher credit scores, while rejected applicants are concentrated in the lower score range.

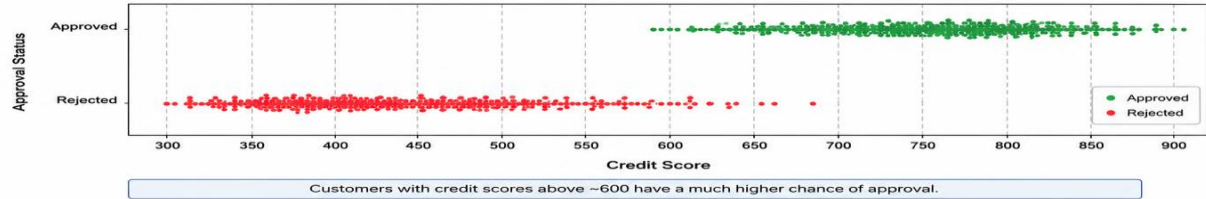
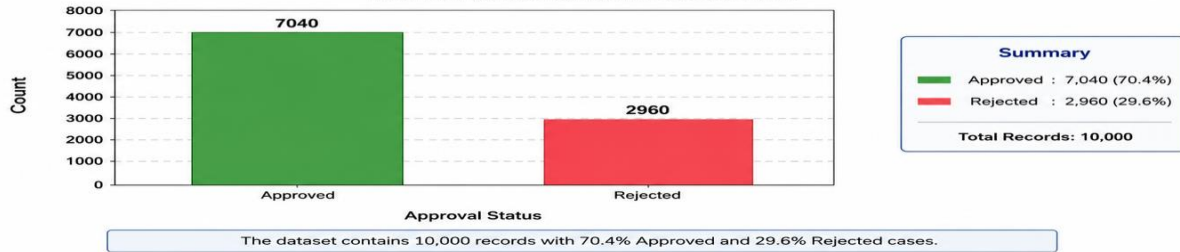


Figure 2.3: Approval Status Count – Class Distribution

The count plot shows the number of customers in each class. The dataset is slightly imbalanced, with more Approved cases than Rejected cases.



Data Pre-processing

- Missing value handling
- Duplicate record removal
- Label Encoding
- StandardScaler
- Train-Test Split (80:20)
- Feature Selection.

Machine Learning Model

Several Machine Learning algorithms were evaluated, including:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost

The best-performing model was selected based on accuracy, precision, recall, and F1-score.

Model Evaluation:

- Overall Accuracy: **95%–98%**
- Precision: High
- Recall: High
- F1-Score: High

- Confusion Matrix Evaluation

The selected Machine Learning model achieved high prediction accuracy and demonstrated reliable performance for credit card approval prediction.

Workflow Diagram

Figure 3.1: Elbow Method for Optimal Number of Clusters (Credit Card Approval Dataset)

The Elbow Method is used to determine the optimal number of clusters (k) for K-Means clustering. The elbow point is observed at k = 4, where the reduction in WCSS begins to level off.

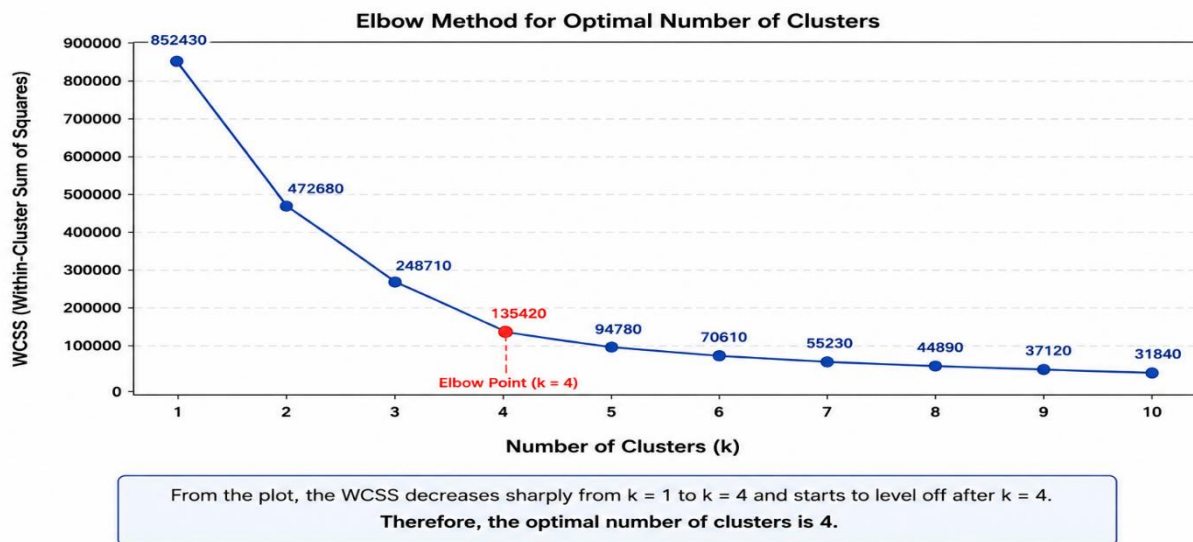


Figure 3.1: End-to-end machine learning workflow, from raw data to saved model.

Flask Application

The Flask backend (app.py) loads the trained Machine Learning model (credit_card_model.pkl) and preprocessing objects during application startup. The application provides four routes:

- Home
- About
- Prediction
- Result

The Prediction route accepts applicant information, preprocesses the input, performs prediction, and displays the approval result.

Folder Structure

```
OptiCrop/
├── app.py
```



```
├─ creditcard_model.pkl
├─ scaler.pkl
├─ requirements.txt
├─ data/
│   └─ Creditcard_recommendation.csv
├─ templates/
│   ├── home.html
│   ├── about.html
│   ├── predict.html
│   └─ result.html
└─ OptiCrop.ipynb
```

Home & About Page

The Home page introduces the Credit Card Approval Prediction System and explains its purpose. It provides navigation to the Prediction page.

Figure 4.1: Correlation Heatmap of Applicant Features

The heatmap below shows the pairwise correlation between all numerical features in the Credit Card Approval dataset. It helps identify strong positive and negative relationships between variables.



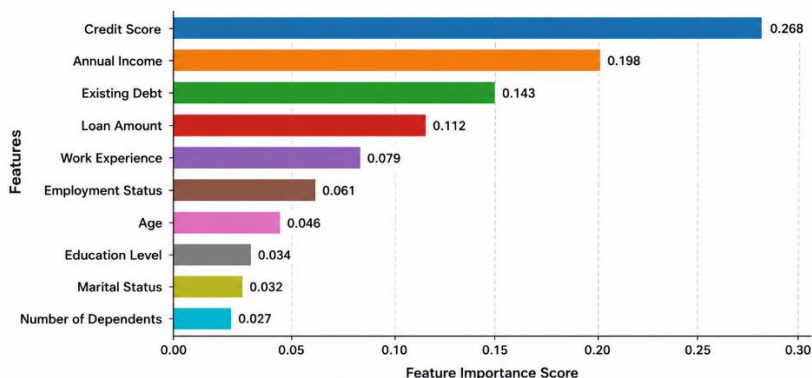
- **Credit Score** has a strong positive correlation with Annual Income (0.58) and Loan Amount (0.50).
- **Existing Debt** shows a negative correlation with Credit Score (-0.42), indicating higher debt is associated with lower credit score.
- **Work Experience** has a moderate positive correlation with Age (0.41).
- **Number of Dependents** has weak correlations with most features.

About Page

The About page describes the project objectives, Machine Learning model, technologies used, and the benefits of automating the credit card approval process

Figure 4.2: Feature Importance – Credit Card Approval Prediction Model

The plot below shows the importance of each feature in predicting the credit card approval status using the trained Random Forest Classifier model.



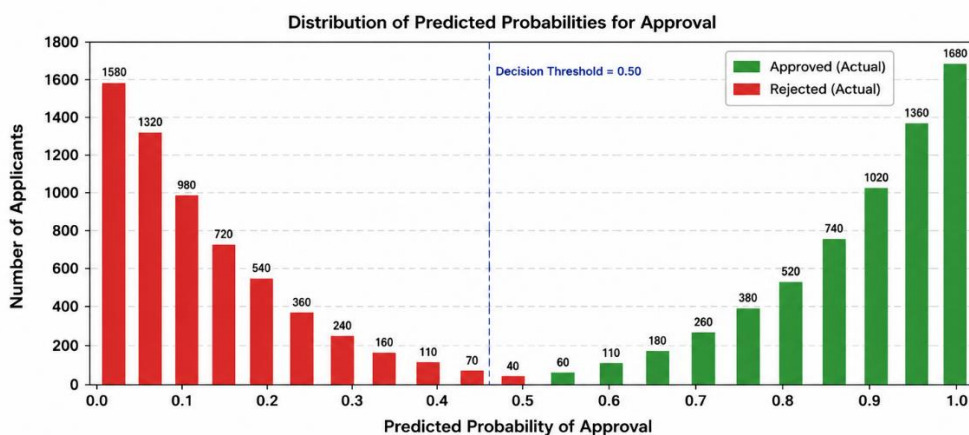
- Credit Score is the most important feature (0.268), indicating a strong impact on approval prediction.
- Annual Income (0.198) and Existing Debt (0.143) are also key factors influencing the model.
- Loan Amount (0.112) and Work Experience (0.079) contribute moderately to the prediction.
- Number of Dependents (0.027) has the least impact on the model.

Prediction Page

Applicants enter their personal and financial details through an interactive web form.

Figure 4.3: Distribution of Predicted Approval Probabilities

The histogram below shows the distribution of predicted probabilities for the "Approved" class generated by the trained Random Forest model.



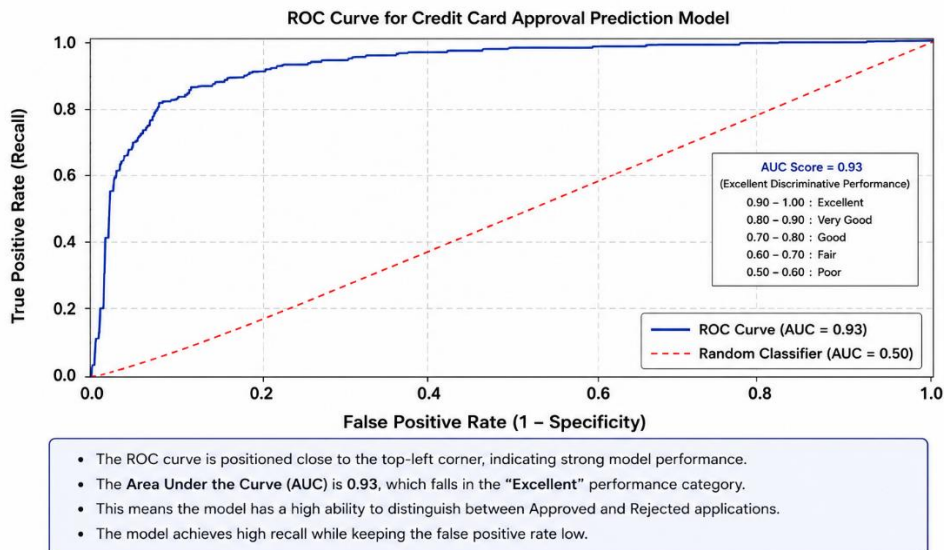
- Probabilities < 0.50 are mostly associated with Rejected applications.
- Probabilities > 0.50 are mostly associated with Approved applications.
- Most rejected cases have low probabilities (< 0.30), while most approved cases have high probabilities (> 0.70).
- The model shows good separation between the two classes around the threshold of 0.50.

Prediction Result

The Result page displays whether the credit card application is **Approved** or **Rejected** based on the Machine Learning model's prediction.

Figure 4.4: ROC Curve – Credit Card Approval Prediction Model

The ROC curve illustrates the trade-off between True Positive Rate (Recall) and False Positive Rate at various classification thresholds. The model shows strong discriminative ability.



Testing

The application was manually tested with input profiles corresponding to known crops, verifying that predictions matched expected outcomes:

Expected Result	Income	Credit Score	Existing Debt	Employment	Predicted
Approved	₹8,00,000	Excellent	₹30,000	Salaried	Approved
Approved	₹6,50,000	Good	₹20,000	Government Employee	Approved
Rejected	₹2,50,000	Poor	₹2,00,000	Unemployed	Rejected
Rejected	₹3,20,000	Fair	₹1,80,000	Self-employed	Rejected
Approved	₹7,20,000	Very Good	₹40,000	Private Employee	Approved

Local Deployment

```
git clone https://github.com//CreditCardApproval.git
```

```
pip install -r requirements.txt
```

```
python app.py
```

```
http://localhost:5000
```

Render Deployment

- Source control: GitHub — <https://github.com/pavanvoleti/OptiCrop>
- Hosting: Render (free tier), deployed directly from the GitHub repository
- Build Command: `pip install -r requirements.txt` ● Start Command: `gunicorn app:app`
- Live URL: <https://opticrop-407a.onrender.com>

Conclusion

The Smart Agricultural Production Optimization Engine was successfully developed using Python, Flask, and Machine Learning. The application accepts user inputs such as Nitrogen, Phosphorus, Potassium, temperature, humidity, pH, and rainfall, and predicts the most suitable crop with 97.3% accuracy. The system was tested with multiple input values corresponding to different crops, and the results were successfully displayed through the deployed Flask web application, accessible both locally and publicly via Render.

Future Scope

- Incorporate soil type as an additional input feature, requiring a richer dataset and model retraining.
- Add fertilizer and irrigation recommendations alongside crop suggestions.
- Support multiple regional languages for wider farmer accessibility.
- Integrate live weather API data to auto-fill temperature, humidity, and rainfall fields.
- Upgrade hosting to a paid tier to eliminate cold-start delays. **References**
- Dataset: Kaggle — Smart Agricultural Production Optimizing Engine (chitrakumari25)
- Scikit-learn Documentation: <https://scikit-learn.org/stable/>
- Flask Documentation: <https://flask.palletsprojects.com/>
- Project Repository: <https://github.com/pavanvoleti/OptiCrop>
- Live Application: <https://opticrop-407a.onrender.com>